The Islamia University of Bahawalpur
Department of Information Technology

INFT-4041 · BSIT · 5th Semester · Fall 2026

E-Commerce Application Development

Chapter

7

Product Variations, Options and Uploads

Part II — Standing the Shop Up

Taught in	Week 5
Laboratory	Option combinations, and what an upload field accepts
Assessment	**Quiz 1 (Ch 1–5)**

This is a class extract. It is *Chapter 7* of the unified handout, printed on its own so that a week's reading is a week's reading. Page numbers restart at 1 and are local to this extract; chapter, figure and table numbers are the ones the full volume gives them, so a reference you follow here lands in the same place there.

Cross-references to other chapters print as their numbers in the full volume. For the whole course in one file — front matter, the HEC tracing table, all twenty-three chapters and all seven appendices — see `ECAD_Fall2026_Handout.pdf`.

Dr. Mustafa Hameed · Department of Information Technology

Generated from `parts/` by `makechapters.py`; do not edit this PDF's source by hand

Chapter 7

Product Variations, Options and Uploads

Learning Outcomes

By the end of this chapter you will be able to:

- **Distinguish** an option from a variant, and **say** which of the two a given requirement needs.
- **Explain** why options do not store the combinations they imply, and **state** what that makes impossible.
- **Configure** option sets on a product and **predict** the number of rows each one creates.
- **Describe** what a shop must check before accepting an uploaded file, and **identify** which of those checks a client can defeat.
- **Test** your own shop's upload handling and **report** what it accepts.

Before You Start

Chapter 6: the eighteen product tables, and the habit of looking at what a screen wrote. Your shop should still have its demonstration catalogue; `seed/catalogue.py -restore` puts it back if not.

Key Terms in This Chapter

option • option value • variant • SKU • modifier • price prefix • combinatorial explosion • stock keeping • required option • file upload • allow-list • MIME type • content sniffing • stored cross-site scripting

7.1 Two Ways to Say "Comes in Three Colours"

A shirt in three colours and four sizes. Twelve things a customer might buy. How many rows is that?

The honest answer is that it depends on a decision nobody will prompt you to make, and that the two answers have different consequences for the rest of the shop's life.

Option and Variant

An **option** is a choice attached to one product, whose values *modify* it. The shirt is one product; "Large" adds Rs. 200 and "Red" adds nothing. The twelve combinations are **not stored anywhere** — they are computed when a customer picks.

A **variant** is a product in its own right, linked to a master. "Red Large Shirt" is a row in `oc_product`, with `master_id` pointing at the parent, its own model number, its own stock and its own price. Twelve combinations are twelve rows.

OpenCart 4 supports both — `oc_product` carries `master_id`, `variant` and `override` columns alongside the older option tables. The admin panel shows options first, so options are what most shops end up with, including shops that needed variants.

7.1.1 What options actually store

Two tables do the work, and a third names the choices globally.

Table	What a row means
`oc_option`	A choice that exists in the shop at all — "Size", "Colour" — with its `type`: `select`, `radio`, `checkbox`, `text`, `textarea`, `date`, `time`, `datetime`, `file`
`oc_option_value`	One possible value of it: "Large". Shared across every product that uses the option
`oc_product_option`	*This* product offers *that* option, and whether it is `required`
`oc_product_option_value`	This product offers that value, and what choosing it does: `price` with a `price_prefix` of `+` or `-`, plus `quantity`, `weight` and `points` adjustments

⚠ What options make impossible

Look at where `quantity` lives: on `oc_product_option_value`, which is *per option value*, not per combination.

So the shop can know it has 40 Large shirts and 30 Red shirts. It **cannot** know how many Red Large shirts it has, because there is no row for that — and no amount of configuration will add one. Three consequences follow, and all three are discovered late:

- **No stock per combination.** You will oversell the popular combination and sit on the unpopular one.
- **No SKU or barcode per combination**, so the shop cannot speak to a warehouse system that works in SKUs — which is all of them.
- **No price history or reporting per combination.** "How many Red Large did we sell?" has no answer.

If any of those three matter, you need variants, and you need to know that *before* the catalogue is built. Migrating two thousand option-based products to variants is not a setting; it is a project.

Which one does this requirement need?

A test that settles most cases in one question: **does the shop need to count, price or identify the combination on its own?**

Options are right when the choice does not change what you hold in stock: an engraving message, a delivery date, a gift-wrap tick, a cable length that you cut to order. Also when the combination genuinely shares stock — which is rarer than people assume.

Variants are right whenever the combination is a thing in a box on a shelf. Clothing, shoes, anything with a barcode. If a warehouse person could pick up exactly one combination and scan it, it is a variant.

The common mistake is to choose options because the admin panel offers them first and they are quicker to set up, and to discover in month nine — during the first real stock-take — that the shop has never known what it holds.

7.2 What Each Choice Costs

Options are cheap to store and variants are not. That much is obvious. What is worth measuring is *how fast* the difference grows, because it decides whether a catalogue is manageable.

Measured: Options Against Variants

Produced by `code/m07_variations.py` on the machine in Appendix A. Option sets are attached to one product and the product page is requested seven times, median taken; the variant column is the number of `oc_product` rows the same catalogue would need.

Options	Combin.	Value rows	Page	KB	As variants
3	3	3	87 ms	30.0	4 rows
3×4	12	7	104 ms	30.7	13 rows
3×4×5	60	12	101 ms	31.3	61 rows
6×8×5	**240**	**19**	103 ms	31.3	**241 rows**

Options grow by addition; variants grow by multiplication. Three option sets of 6, 8 and 5 values are **19 rows** — $6+8+5$ — and express 240 combinations. The same catalogue as variants is **241 product rows**, each with its own description row, its own store row and its own category rows. Add a sixth colour and options gain one row while variants gain forty.

And the product page does not care. From 3 combinations to 240, the page stayed at about 100 ms and grew by **1.3 kilobytes** — because the page renders the *choices*, not the combinations. Nineteen values in three dropdowns, whatever the 240 products those values imply.

So the cost of variants is not page speed; it is catalogue management. Two hundred and forty-one rows that must each be priced, stocked, described, photographed and kept in step. That is the real bill, it is paid by a person rather than a server, and it is the reason shops that should have chosen variants did not.

7.3 Uploads, and What Your Shop Will Accept

One of the nine option types is `file`. Switch it on and a customer can send you a file — a logo to print, a document to quote against, a photograph to frame.

You have now accepted arbitrary bytes from a stranger, and everything that follows is about not regretting it.

What OpenCart checks, in order

From `catalog/controller/tool/upload.php` in your own container. Read it; it is forty lines.

1. **The filename is sanitised** — everything outside `[a-zA-Z0-9.␣+]` is stripped, then `basename()` removes any path. This defeats `../../etc/passwd` and it is solid.
2. **The length is checked**, 3 to 64 characters.
3. **The extension is checked against an allow-list** (`config_file_ext_allowed`). An allow-list is the right shape: anything not named is refused.
4. **The MIME type is checked against a second allow-list** (`config_file_mime_allowed`).

5. **The stored name is randomised** — the file is saved as the name plus a 32-character token, so it cannot be guessed, and the real name is kept in the database.

Check 4 is worth almost nothing, and it is not a bug in OpenCart

The MIME type it compares is `$_FILES['file']['type']` — and that value is **sent by the browser**. It is a claim made by the client about its own file. Anyone using anything other than a browser sets it to whatever passes.

The default allow-list makes this worse by including `application/octet-stream`, which means "unidentified binary" — so the honest answer for almost any file is already on the list.

This is not a criticism of OpenCart so much as a lesson about where a check must run. A check performed on data the attacker controls is not a check. The extension allow-list is real because the server derives it from the name it has already sanitised. The MIME check is advisory, and the only server-side equivalent is to inspect the file's actual contents — `finfo_file()` in PHP, which reads the magic bytes.

Two defaults on your shop, which you can verify in five minutes

Both of these are true of the stock install in Chapter 5, and both are measured in Chapter 16.

The allow-list includes `svg`. An SVG is not a picture; it is an XML document, and it may contain `<script>`. A browser asked to display one from your origin will run it. The default list also carries `eps` and `ps`, which are PostScript — a programming language — and `msi`, `cab` and `rar`.

Uploads are served over HTTP. They land in `system/storage/upload/`, which is *inside* the web root. Requesting a file there returns it: a text file comes back as `text/plain` and an SVG as `image/svg+xml`. The directory cannot be listed — there is an empty `index.html` — and names carry a 32-character token, so this is not trivially exploitable. But the protection is secrecy of the name, not inaccessibility of the directory, and Apache is configured with `AllowOverride None`, so the `.htaccess` OpenCart ships cannot change it.

What to do about it is Chapter 16's subject. The short version: remove `svg` from the list unless you need it, serve uploads through a script that sets `Content-Disposition: attachment` rather than from the filesystem, and keep the storage directory outside the web root.

Two size limits, and the smaller one wins

`config_file_max_size` is a setting in the admin panel. PHP has its own, `upload_max_filesize`, in `code/shop/php.ini`, and this course sets it to 8 MB.

If the admin setting is larger, it has no effect: PHP refuses the upload before any OpenCart code runs, and the error the customer sees is unhelpful. Whenever a limit appears not to work, look for the second limit — there is nearly always a second limit, in the web server, the runtime or the proxy, and the smallest one is the one in force.

Common Pitfalls

- **Choosing options when the requirement needs variants.** The symptom arrives at the first stock-take, and the fix is a migration.
- **Believing option `quantity` tracks combinations.** It tracks one value. Red Large and Red Small share Red's stock.
- **Trusting the uploaded MIME type.** The client supplies it.
- **Allowing `svg` without thinking about it.** It is script that renders as a picture.

- **Setting a size limit in one place only.** The smaller of the admin setting and `upload_max_filesize` wins.
- **Making every option `required`.** The customer cannot buy without choosing, and a required text option with no sensible default costs sales.
- **Forgetting that option values are global.** Editing "Large" edits it for every product that uses it.

Try It Yourself: Options, variants, and what your shop will swallow

```
# --- 1. Look at a product that already has every option type ---------
# Product 42 in the demo catalogue carries all nine. Open it in the
# admin panel: Catalog > Products > edit > Option.
cd code
docker compose exec db mariadb -ushop -pshoppw opencart

# --- 2. Count what one option set costs -----------------------------
# Add a Size option with three values, through the admin panel. Then
# count the rows it created. Predict the number before you look.
#   SELECT COUNT(*) FROM oc_product_option       WHERE product_id = 42;
#   SELECT COUNT(*) FROM oc_product_option_value WHERE product_id = 42;

# --- 3. Find the row that does not exist ----------------------------
# Set stock on two option values. Then try to write a query that
# answers "how many Red Large do we hold?". You cannot. Write down,
# in one sentence, why -- that sentence is the whole of section 7.1.

# --- 4. Measure options against variants ----------------------------
python m07_variations.py

# --- 5. Read the upload controller ----------------------------------
# Forty lines. Find the five checks, and find the one the client
# controls.
docker compose exec shop \
  cat /var/www/html/catalog/controller/tool/upload.php

# --- 6. Find out what your shop allows ------------------------------
docker compose exec db mariadb -ushop -pshoppw opencart -e \
  "SELECT \`key\`,\`value\` FROM oc_setting \
    WHERE \`key\` LIKE 'config_file%';"

# --- 7. Prove the upload directory is served ------------------------
# Put a file there yourself and fetch it. This is YOUR container; see
# the scope statement in Appendix A before you try this anywhere else.
docker compose exec shop sh -c \
  'echo hello > /var/www/html/system/storage/upload/probe.txt'
curl -i http://localhost:8090/system/storage/upload/probe.txt
docker compose exec shop rm /var/www/html/system/storage/upload/probe.txt

# --- 8. Decide, and write it down -----------------------------------
# For the shop you described in Chapter 4: options or variants? One
# paragraph, with the stock question answered explicitly. Chapter 23
# marks you on whether the catalogue matches this decision.
```

What to notice. Step 3 is the one that teaches. Everybody believes option stock tracks combinations until they try to write the query, and the minute spent failing to write it is worth more than this chapter's prose.

Checkpoint — Test Yourself

1. A shop sells mugs with a customer-supplied photograph printed on them, in two sizes. Which choices are options and which are variants, and why? Say what the shop must hold in stock.
2. A product has option sets of 4, 3 and 2 values. How many rows in `oc_product_option_value`, and how many `oc_product` rows would the same catalogue need as variants?
3. An attacker uploads a file named `logo.svg` containing `<script>`, sending `Content-Type: image/png`. Which of OpenCart's five checks does it pass, which does it fail, and what would have to be true for this to harm a visitor?

Chapter Summary

- An option modifies one product; a variant is a product of its own with `master_id` pointing at a master. OpenCart 4 supports both and shows options first.
- Options store choices, not combinations. Stock, price and weight sit on the option *value*, so there is no row for "Red Large" and no query can produce one.
- That makes three things impossible: stock per combination, a SKU per combination, and reporting per combination. If any matters, use variants, and decide before the catalogue is built.
- The test: could a warehouse person pick up exactly this combination and scan it? Then it is a variant.
- Measured: option sets of 6, 8 and 5 values are 19 rows and express 240 combinations; the same catalogue as variants is 241 product rows. Options grow by addition, variants by multiplication — and the page time is unaffected either way, because the page renders choices.
- The real cost of variants is catalogue management by a person, not server time.
- A `file` option accepts arbitrary bytes from a stranger. OpenCart sanitises the name, checks the extension against an allow-list and randomises the stored name — all sound. Its MIME check is not, because the client supplies the MIME type.
- On the stock install `svg` is allowed and the upload directory is served over HTTP. The protection is an unguessable name, not an unreachable directory. Chapter 16 fixes both.

Review Questions

1. (MCQ) Stock quantity for an option lives on: (a) `oc_product`; (b) `oc_product_option`; (c) `oc_product_option_value`; (d) a combination table.

2. (MCQ) Three option sets of 5, 4 and 3 values imply how many combinations, and cost how many `oc_product_option_value` rows? (a) 12 and 12; (b) 60 and 12; (c) 60 and 60; (d) 12 and 60.

3. (Short) Give the one-question test for options against variants, and apply it to a shop selling curtains cut to a customer-specified width.

4. (Short) Why is checking `$_FILES['file']['type']` not a security control, and what is the server-side equivalent?

5. (Short) An administrator raises the upload size limit in the admin panel and large files are still refused. Explain, and say where else to look.

6. (Applied) A clothing shop has 300 styles, each in 5 sizes and 4 colours. Compute the row counts for options and for variants across `oc_product` and `oc_product_description`. Then make a recommendation, naming the single requirement that decides it.

7. **(Applied)** Design the upload handling you would write for a shop that must accept customer artwork. State every check, say which side it runs on, say what each one defends against, and say what you would do with the file after accepting it so that serving it later is safe.